MediFlow — Análise de Requisitos

Equipe: Hackathon ONE G10 Equipo 15 | **Versão:** 1.1 | **Data:** 16/10/2026

Como usar este documento: leia as seções 1 a 4 para entender o projeto; use as seções 5 a 8 como referência durante o desenvolvimento; a seção 12 registra as decisões já tomadas e as que o time ainda precisa tomar. Este documento é a base da Especificação Funcional.

1. Visão geral

O que é o MediFlow. Um agente de IA que recebe documentos clínicos (receitas, laudos, guias, atestados), descobre sozinho que tipo de documento é, extrai os dados importantes e envia o documento para o setor certo do hospital. Quando o agente não tem certeza, ou quando o caso é urgente, ele avisa uma pessoa.

A solução MediFlow deve resolver desafios complexos de visão multimodal, extração estruturada de dados clínicos, lógica de decisão condicional e tratamento de casos ambíguos ou urgentes (como dados ilegíveis, prescrições de alto risco ou solicitações de urgência).

Objetivo do hackathon. Entregar um MVP (versão mínima funcional) do agente, com demonstração de três cenários: rotina, urgência e caso ambíguo.

Formações ONE relacionadas. Indispensáveis: Engenharia de Agentes e Automação e Oracle Cloud Infrastructure (OCI). Reforço: Desarrollo y Orquestación con IA Generativa.

Stack adotada pelo time. LLM: Gemini · Orquestração: LangGraph · Validação: Pydantic · Interface: React + FastAPI · Armazenamento: OCI Object Storage.

Como o agente funciona, em 6 passos

1. Receber o documento clínico ou administrativo nos formatos suportados. (PDF, imagem, texto ou JSON)
2. Classificar o tipo, a especialidade e a prioridade. (ex.: Receituário Médico, Laudo de Exame Laboratorial/Imagem, Guia de Solicitação de Procedimento...)
3. Extrair entidades e os dados clínicos (paciente, médico, diagnóstico, medicamentos, exames) com altíssima precisão.
4. Avaliar a consistência, potenciais conflitos e a nota combinada de confiabilidade (classificação + extração).
5. Rotear para o destino correto em casos padrão, ou para revisão humana se houver dúvida (nota combinada < 0.85, por padrão).
6. Guardar o documento e a decisão no OCI Object Storage e devolver o resultado em JSON.

2. Glossário

Termo	Significado
MVP	Produto mínimo viável: a menor versão que já funciona e cumpre o essencial.
Agente autônomo	Programa que toma decisões e executa etapas sozinho, usando uma IA.
LLM	Modelo de linguagem que lê e interpreta textos e imagens. No MediFlow, o modelo adotado é o Gemini.
Multimodal	Capaz de entender mais de um tipo de entrada, como texto e imagem.
Human-in-the-Loop (HITL)	“Humano no circuito”: uma pessoa revisa os casos em que a IA não tem certeza.
Score de confiança	Nota de 0 a 1 que indica o quanto o sistema confia no próprio resultado.

Termo	Significado
Nota combinada	Nota final de 0 a 1 que junta a confiança da classificação e a da extração. É ela que é comparada com o limiar.
Limiar de confiança	Valor mínimo da nota combinada para o documento seguir sem revisão humana. Padrão: **0.85**. Abaixo disso, o documento vai obrigatoriamente para a Fila de Revisão Humana.
Grafo de decisão	Fluxo com etapas (nós) e caminhos condicionais: se urgente, vai para A; se ambíguo, vai para B.
LangGraph / n8n	LangGraph (Python) monta o grafo de decisão do MediFlow. O n8n é uma ferramenta de fluxos visuais, citada apenas nos alertas opcionais (RF-D5).
Pydantic / JSON	Biblioteca Python que valida tipos / Formato de texto estruturado para dados.
React / FastAPI / Streamlit	React: biblioteca para montar a tela. FastAPI: framework Python para a API. Streamlit: ferramenta Python para painéis rápidos (painel do gestor).

3. Partes interessadas

Papel	O que precisa do sistema
Gestor hospitalar / operadora (cliente)	Reduzir horas de digitação, erros e glosas; acelerar autorizações; calibrar o limite de confiança para revisão.
Equipe administrativa	Documentos já classificados e com os dados extraídos, sem redigitar.
Auditor clínico	Fila clara dos casos duvidosos, com o motivo da dúvida, para aprovar ou rejeitar.
Equipe de plantão médico	Alerta imediato quando há achado crítico.
Farmácia e Auditoria de autorizações	Receber só os documentos do seu tipo, já estruturados.
Avaliadores do hackathon	Checklist cumprido, demonstração de 3 cenários e documentação completa.
Time de desenvolvimento	Requisitos claros e divisão de trabalho.

4. Escopo

Dentro do escopo (MVP obrigatório)

- Receber documentos em PDF, imagem, texto ou JSON.
- Classificar, extrair, avaliar a confiança e rotear usando LLM multimodal (Gemini).
- Grafo de decisão condicional (LangGraph) com tratamento de urgência e ambiguidade.
- Limiar de confiança padrão de 0.85, guardado em variável de configuração: nota combinada abaixo dele vai obrigatoriamente para revisão humana.
- Persistência no OCI Object Storage (Always Free).
- Interface de triagem em React e API REST em FastAPI, com dados validados por Pydantic.
- Repositório Git, README com diagrama e demonstração de 3 cenários.

Diferenciais (opcionais, só depois do MVP)

- Configuração externa pelo gestor (limites, regras e destinos sem mexer no código), começando por um slider interativo no Streamlit para calibrar o limiar de confiança.
- Deploy na OCI Compute (VM Always Free).
- Leitura de receitas manuscritas e laudos escaneados (OCR e visão computacional).
- Painel de auditoria HITL em Streamlit (aprovar/rejeitar com um clique) e Alertas em tempo real.

5. Requisitos funcionais

Prioridade: O = obrigatório (MVP), D = diferencial. O símbolo + marca requisitos derivados pela análise.

5.1 Ingestão

ID	Requisito	Prior.
RF-01	Receber documentos por endpoint (FastAPI) ou tela de envio (React).	O
RF-02	Aceitar PDF (digital e digitalizado), imagem e texto/JSON.	O
RF-03 +	Validar formato e campos obrigatórios da entrada, rejeitando com mensagem clara.	O
RF-04	Ler o conteúdo do documento com LLM multimodal (Gemini).	O

5.2 Classificação

ID	Requisito	Prior.
RF-05	Identificar a categoria: Receituário Médico, Laudo de Exame Laboratorial/Imagem, Guia, Relatório de Alta/Internação e Atestado Médico.	O
RF-06	Identificar a especialidade médica relacionada ao documento (campo *especialidade* da saída).	O
RF-07	Classificar o nível de prioridade do documento: no mínimo “Urgente” e “Rotina” (campo *nivel_prioridade* da saída).	O

5.3 Extração

ID	Requisito	Prior.
RF-08	Extrair a identificação do paciente (nome, idade).	O
RF-09	Extrair o profissional solicitante: nome e CRM (formato número-UF).	O
RF-10	Extrair a hipótese diagnóstica ou diagnóstico principal e o CID-10.	O
RF-11	Extrair medicamentos e dosagens, quando houver.	O
RF-12	Extrair os exames solicitados ou realizados.	O
RF-13	Extrair o indicador de urgência.	O
RF-14	Entregar os dados em JSON validado com Pydantic.	O
RF-15 +	Nunca inventar dado ausente: o campo faltante fica vazio e é sinalizado.	O

5.4 Decisão e roteamento

ID	Requisito	Prior.
RF-16	Avaliar consistência e conflitos (ex.: CID incompatível com diagnóstico, dose incomum).	O
RF-17 / 18	Calcular a nota combinada de confiabilidade (classificação + extração), de 0 a 1 / Detectar urgências e alto risco.	O
RF-19 / 20	Grafo de decisão (LangGraph) com nós condicionais / Roteamento para as filas específicas.	O
RF-21	Enviar para revisão humana quando houver ambiguidade, dado faltante ou **nota combinada < 0.85**. O 0.85 é o limiar padrão, lido de uma variável de configuração; abaixo dele o envio para a Fila de Revisão Humana é obrigatório. Nota igual ou maior que 0.85 segue o fluxo normal, se não houver outro motivo de revisão ou urgência.	O
RF-22 a 24 +	Gerar notificação p/ urgentes, registrar justificativa e tratar exceções com fallback humano.	O

5.5 Resultado e histórico

ID	Requisito	Prior.
RF-25	Retornar resposta estruturada (status, dados extraídos, decisão, armazenamento).	O
RF-26	Disponibilizar o histórico de triagem em JSON.	O
RF-27	Oferecer interface de triagem (React) e API REST (FastAPI) para visualizar os fluxos.	O

5.6 Armazenamento (OCI) e 5.7 Diferenciais

ID	Requisito	Prior.
RF-28 a 30	Salvar no OCI (Always Free), separar buckets por status, e informar caminho na resposta.	O
RF-D1	Configuração externa pelo gestor, sem mexer no código. O limiar de confiança (padrão 0.85) fica em variável de configuração (isso já faz parte do MVP) e o diferencial é o **slider interativo no Streamlit** para o gestor calibrar esse limite. Regras e destinos customizáveis também entram neste diferencial.	D
RF-D2 a D5	Deploy na OCI Compute, OCR, painel HITL em Streamlit e alertas n8n.	D

6. Requisitos não funcionais

ID	Categoria	Requisito	Origem
RNF-01	Precisão	Extração com altíssima precisão; erro clínico é falha grave.	Brief
RNF-02	Segurança	Nenhuma urgência tratada como rotina; na dúvida, revisão.	Derivado
RNF-03	Confiabilidade	Falhas não derrubam o sistema nem perdem documentos.	Brief
RNF-04	Rastreabilidade	Toda decisão tem justificativa e fica registrada no histórico.	Brief
RNF-05	Custo	Usar exclusivamente recursos Always Free da OCI.	Brief
RNF-06	Padronização	Saídas sempre em JSON válido e tipado (Pydantic).	Derivado
RNF-07	Agilidade	Resposta rápida para não atrasar casos urgentes.	Brief
RNF-08	Privacidade	Apenas dados fictícios; nenhuma chave/segredo no Git.	Brief
RNF-09	Manutenção	Código modular, commits documentados e README atual.	Brief

ID	Categoria	Requisito	Origem
RNF-10	Flexibilidade	Camada de LLM isolada em módulo próprio, para poder trocar o modelo sem refazer o pipeline (o modelo adotado é o Gemini).	Brief
RNF-11	Usabilidade	Interface simples (tipo, prioridade, destino e justificativa).	Derivado

7. Regras de negócio

ID	Regra
RN-01	Casos padrão (sem ambiguidade e com nota combinada de 0.85 ou mais) seguem direto ao destino, sem intervenção humana.
RN-02	Documento ambíguo, ilegível, inconsistente ou com **nota combinada < 0.85** (limiar padrão) vai obrigatoriamente para a Fila de Revisão Humana.
RN-03	Achado crítico ou prescrição de alto risco vai para a Fila de Urgência Médica com alerta.
RN-04	Todo documento é salvo no OCI conforme seu status (recebido, processado, auditoria).
RN-05	Toda decisão de roteamento traz uma justificativa estruturada.
RN-06	Somente recursos Always Free da OCI podem ser usados.
RN-07 +	Cada documento tem um documento_id único gerado pelo sistema.

Roteamento proposto

Situação	Destino Sugerido
Achado crítico ou urgência	Fila de Urgência Médica, com alerta
Receituário de rotina	Farmácia Hospitalar
Guia de solicitação de procedimento	Auditoria de Autorizações
Laudos, relatórios de alta e atestados de rotina	Prontuário Eletrônico do Paciente
Ambiguidade, dado faltante ou nota combinada < 0.85	Fila de Revisão Humana

8. Dados de entrada e saída (visão geral)

Entrada

Campo	Descrição	Exemplo
documento_id	Identificador único do documento	DOC-CLIN-2026-8942
tipo_arquivo	Formato do arquivo	PDF
documento_texto	Texto do documento	Texto do laudo...
canal_origem	Setor de onde o documento veio	Pronto_Socorro

Saída

Bloco	Campos
status	Situação do processamento (ex.: processado, em_auditoria)
documento_id	Mesmo identificador da entrada
classificacao	tipo_documento, especialidade, nivel_prioridade, score_confianca (nota combinada, comparada com o limiar 0.85)
dados_extraidos	paciente (nome, idade); medico (nome, crm); diagnostico_principal; cid
decisao_roteamento	destino_principal, requer_auditoria_humana, justificativa, notificacao
armazenamento_oci	bucket, caminho_objeto, status_backup

Atenção: o exemplo do brief original não mostra campos para medicamentos, dosagens, exames solicitados, urgência detalhada, score de extração nem conflitos, mas o texto exige essas informações. Esses campos devem constar na Especificação Funcional final.

9. Tecnologias

Camada	Opções permitidas pelo brief	Situação
LLM	Google Gemini (curso), OpenAI, Claude	**Decidido:** Gemini
Orquestração	LangGraph, n8n, Python puro, CrewAI	**Decidido:** LangGraph
Validação de dados	Pydantic ou JSON Schema	**Decidido:** Pydantic
Interface ou API	Streamlit, FastAPI, Flask, n8n, Node.js	**Decidido:** React (interface) + FastAPI (API)
Painel do gestor (diferencial)	Streamlit	Slider do limiar (RF-D1) e painel HITL (RF-D4)
Armazenamento	OCI Object Storage (Always Free)	Obrigatório
Hospedagem	OCI Compute (VM Always Free)	Opcional
Versionamento	Git e GitHub	Obrigatório

10. Entregáveis e critérios de aceite

Entregáveis: Repositório Git com commits bem documentados; README com arquitetura e diagrama de decisão; Notebook/Módulos do pipeline; e Aplicação com endpoint/tela.

Demonstração Obrigatória (3 cenários)

1. Fluxo padrão aprovado, sem intervenção humana.
2. Caso com prioridade de urgência médica.
3. Caso ambíguo direcionado para auditoria humana.

11. Restrições, premissas e riscos

Restrições: usar só OCI Always Free; prazo de 5 semanas. **Premissas:** documentos fictícios em testes; filas simuladas.

Risco	Impacto	Mitigação
Erros do LLM (dado inventado)	Dado clínico incorreto	Validação Pydantic, score e revisão humana.
Urgência não detectada	Risco grave ao paciente	Regras explícitas além do LLM.
Respostas do LLM instáveis	Inconsistências	Testes de consistência (QA William).
Conflitos de código da equipe	Retrabalho	Frentes claras e Pull Requests pequenos.
Escopo grande para 5 semanas	MVP incompleto	Priorizar núcleo; diferenciais por último.
Chaves de API expostas	Falha de segurança	Usar .gitignore e .env; não commitar segredos.

12. Decisões do time e pontos em aberto

Decisão	Situação	Detalhe
LLM principal	**Decidido**	Gemini.
Orquestração	**Decidido**	LangGraph.
Interface e API	**Decidido**	React para a tela de triagem e FastAPI para a API. Streamlit fica só no painel do gestor (diferencial).
Validação de dados	**Decidido**	Pydantic.
Limiar de confiança p/ revisão	**Decidido**	Padrão de 0.85, em variável de configuração. Nota combinada < 0.85 vai obrigatoriamente para a Fila de Revisão Humana. Slider no Streamlit para o gestor calibrar (RF-D1).
Fórmula da nota combinada	**A definir**	Sugestão inicial: usar a menor entre a nota de classificação e a de extração (postura conservadora, alinhada ao RNF-02). Ajustar após testes.
Urgência e ambiguidade simultâneas	Decidido	Urgência prevalece: alerta sempre; revisão só com ambiguidade.
Níveis de prioridade exatos	**A definir**	No mínimo “Urgente” e “Rotina”.
Tempo-alvo de resposta	**A definir**	A definir após o primeiro pipeline rodar.

13. Próximos passos

1. Revisar e validar este documento em equipe (principalmente a seção 12).
2. Elaborar a Especificação Funcional, detalhando cada campo, payload e tratamento de erro.
3. Dividir as frentes de trabalho e abrir Issues estruturadas no GitHub.
4. Usar o Plano de Testes (tests/PLANO_DE_TESTES.md) para balizar o desenvolvimento.